



MATH 2007 Report: Numerical analysis

ZHANG JIAYI

BC309867

Class: MATH2007(001)

Date: 2026/4/20

Contents

1. Abstract

2. Interpolation

2.1 Figure for different interpolation methods

2.2 Reasons for best method

2.3 Following problems solutions

2.3.1 Compute $f(1.2)$ and $f(3.6)$

2.3.2 Another method

2.3.3 Comparison

3. Trapezoidal rule

3.1 Two-point quadrature formula with fixed coefficients

3.2 Two-point quadrature formula with free coefficients

3.3 Compute the integrals

3.4 Following problems

4. $f(x) = e^{-x} - \sin x$

4.1 Find the root of $f(x) = 0$

4.2 Performance Comparison

4.3 Newton's method analysis with different starting points

5. ODE with $h = \frac{1}{10}$

5.1 Modified Euler method

5.2 Trapezoidal method

5.3 Performance analysis

5.4 Improvement for approximate solutions

5.5 Implicit method

5.5.1 Order of the implicit method

5.5.2 Solve the ODE

6. Conclusion

7. Appendix (Coding Python)

7.1 Interpolation

7.2 Numerical integration

7.3 $f(x) = e^{-x} - \sin x$

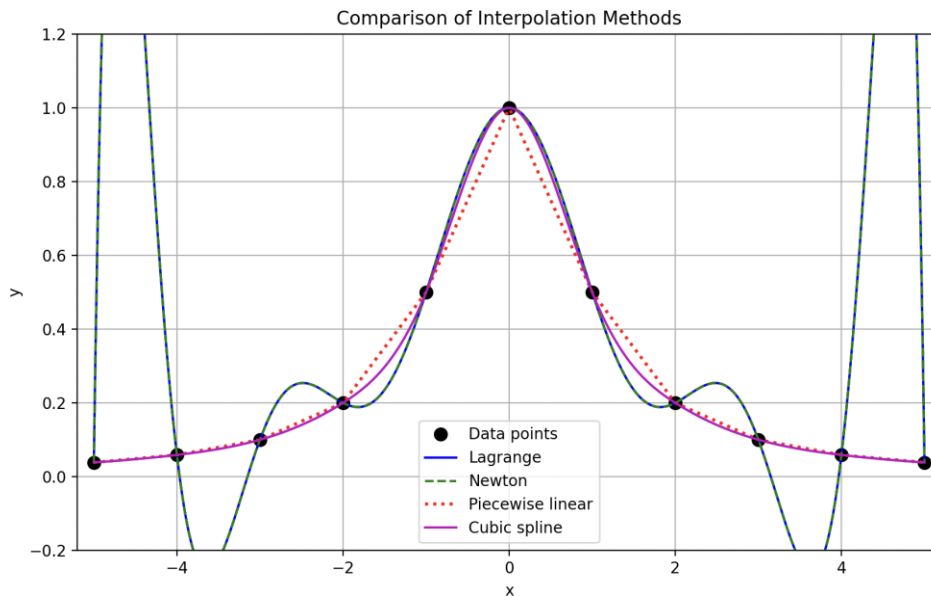
7.4 ODE

1. Abstract

This report provides an in-depth exploration of four core topics in numerical analysis: interpolation algorithms, numerical integration, root-finding for non-linear equations, and numerical solutions for ordinary differential equations (ODEs). First, by comparing Lagrange, Newton, and cubic spline interpolations, the Runge phenomenon in high-degree polynomials under equidistant nodes is analyzed, verifying the advantages of cubic splines in ensuring smoothness and stability. Second, a two-point quadrature formula with maximum algebraic precision is derived and constructed, utilizing the Jacobi weight function to effectively handle numerical integration with weight functions. In the root-finding section, the convergence efficiency and reliability of the bisection method, Newton's method, and the secant method are evaluated. Finally, addressing ODEs with significant stiffness characteristics, various explicit and implicit algorithms are compared, demonstrating that implicit schemes offer greater robustness and accuracy when dealing with stiff problems.

2. Interpolations

2.1 Figure for different interpolation methods



2.2 Reasons for best method

The best method is cubic spline, reasons are following:

1. It guarantees the continuity of the first and second derivatives of the interpolation function, resulting in smooth results.
2. Given the endpoint derivatives, it effectively suppresses the Runge phenomenon, avoiding severe oscillations of higher-order polynomials at the boundaries.
3. While piecewise linear interpolation is non-oscillatory, its derivatives are discontinuous, making it unsuitable for applications requiring smoothness.
4. Lagrange/Newton interpolations are essentially the same; higher-order polynomials interpolated on this equidistant node set are unstable, resulting in large errors.

2.3 Solutions for following problems

2.3.1 Compute the $f(1.2)$ and $f(3.6)$ in interpolation

methods:

$X=1.2$:

	Exact value	Lagrange	Newton	Piecewise	Cubic
	0.4098360656	0.3708327529	0.3708327529	0.440000000	0.4011568442
Error		3.90e-02	3.90e-02	3.02e-02	8.68e-03

$X=3.6$:

	Exact value	Lagrange	Newton	Piecewise	Cubic
	0.07163323	–	–	0.075294117	0.07105372
	78	0.254602669	0.254602669	6	34
		1	1		
Error		3.26e-01	3.26e-01	3.66e-03	5.80e-04

2.3.2 Choose one method

Choose piecewise linear interpolation. The reasons are as following:

1. Simple and quick, requiring no solution of linear equations or calculation of higher-order polynomials.
2. With uniform data points and a smooth function variation within the interval, linear interpolation provides sufficient accuracy.
3. For higher accuracy, cubic splines can be used, but the calculations are slightly more complex. This problem does

not require extremely high accuracy; linear interpolation suffices.

2.3.3 Comparison for interpolation methods:

Analysis of Table 2.3.1:

1. $x = 1.2$:

The cubic spline had the smallest error (approximately 0.0087), performing best.

The piecewise linear error was approximately 0.0302, slightly larger than the cubic spline but still within an acceptable range.

The Lagrange/Newton error was approximately 0.0390, worse than the piecewise linear error, indicating that the higher-order polynomial began to show significant deviation at $x = 1.2$ (near the center-right of the interval).

2. $x = 3.6$:

The cubic spline error is only 5.8×10^{-4} , making it the most accurate.

The piecewise linear error is approximately 3.7×10^{-3} , still relatively small, better than the cubic spline but not by much. Lagrange/Newton produces a negative value (–0.2546), with an error as high as 0.326, indicating complete distortion. This is because $x = 3.6$ is close to the right

endpoint ($x = 5$), where high-order polynomial interpolation experiences severe Runge oscillations.

In conclusion:

Cubic spline interpolation yielded the highest accuracy at both test points (especially at $x = 3.6$, where the error is less than 10^{-3}), and without oscillations, making it the most reliable method.

Piecewise linear interpolation, while simple, had a larger error than cubic spline interpolation at $x = 1.2$ and a slightly larger error at $x = 3.6$, but was generally stable without negative values or other anomalies.

Lagrange/Newton polynomial interpolation (essentially the same) performed well within the interval but failed completely near the boundaries ($x = 3.6$), and is not recommended for high-order interpolation at equidistant nodes.

Therefore, for this dataset, cubic spline interpolation is the best choice, piecewise linear interpolation can be used as a simple alternative, and high-order polynomial interpolation should be avoided.

3. Trapezoidal rule

3.1 Two-point quadrature formula with fixed coefficients

$$\int_0^1 f(x) dx \approx \frac{1}{4}f(x_0) + \frac{3}{4}f(x_1)$$

For $f(x) = 1$: The left-hand side integral equals 1, and the right-hand side equals $\frac{1}{4} + \frac{3}{4} = 1$, so the equation holds trivially.

For $f(x) = x$: The left-hand side integral equals $\frac{1}{2}$, so we have

$$\frac{1}{4}x_0 + \frac{3}{4}x_1 = \frac{1}{2}$$

Simplifying gives:

$$x_0 + 3x_1 = 2$$

For $f(x) = x^2$: The left-hand side integral equals $\frac{1}{3}$, so we have

$$\frac{1}{4}x_0^2 + \frac{3}{4}x_1^2 = \frac{1}{3}$$

Simplifying gives:

$$x_0^2 + 3x_1^2 = \frac{4}{3}$$

Substitute $x_0 = 2 - 3x_1$ into the second equation.

Expanding and simplifying yields the quadratic equation:

$$9x_1^2 - 9x_1 + 2 = 0$$

Solving gives $x_1 = \frac{1}{3}$ or $x_1 = \frac{2}{3}$.

If $x_1 = \frac{1}{3}$, then $x_0 = 1$, which violates the condition $x_0 \leq x_1$, so this solution is discarded.

If $x_1 = \frac{2}{3}$, then $x_0 = 0$, which satisfies the ordering requirement.

Thus, the nodes are: $x_0 = 0$, $x_1 = \frac{2}{3}$

This quadrature rule is exact for all quadratic polynomials but not for cubic polynomials, so its algebraic accuracy is 2

3.2 Two-point quadrature formula with free coefficients

$$\int_0^1 f(x) dx \approx A_0 f(x_0) + A_1 f(x_1)$$

A 2-node quadrature rule can achieve a maximum algebraic accuracy of 3, meaning it must hold exactly for $f(x) = 1, x, x^2, x^3$. This gives the system of equations:

$$\begin{aligned} A_0 + A_1 &= 1 \\ A_0 x_0 + A_1 x_1 &= \frac{1}{2} \\ A_0 x_0^2 + A_1 x_1^2 &= \frac{1}{3} \\ A_0 x_0^3 + A_1 x_1^3 &= \frac{1}{4} \end{aligned}$$

This is the 2-point Gaussian quadrature rule on the interval $[0, 1]$. Solving the system (or transforming the standard Gaussian points from $[-1, 1]$ to $[0, 1]$ gives:

$$x_0 = \frac{3 - \sqrt{3}}{6}, x_1 = \frac{3 + \sqrt{3}}{6}$$

Coefficients:

$$A_0 = \frac{1}{2}, A_1 = \frac{1}{2}$$

This rule is exact for all polynomials of degree at most 3, but not for quartic polynomials, so its algebraic accuracy is 3.

3.3 Compute the following integrals

1. $\int_0^1 x^2 dx$

Exact value : $\frac{1}{3} \approx 0.3333333$

$I1(f): \frac{1}{2} (0^2 + 1^2) = 0.5$ Error = 0.1666667

$I2(f): \frac{1}{4} * 0 + \frac{3}{4} * \left(\frac{2}{3}\right)^2 = \frac{1}{3}$ Error = 0

$I3(f): \frac{1}{2} * (0.2113)^2 + \frac{1}{2} * (0.7887)^2 \approx 0.33333$, Error = 0

$I4(f): 0.2776 * (0.2899)^2 + 0.3891 * (0.8212)^2 \approx 0.285714$, Error = 0.047619

2. $\int_0^1 x^{\frac{3}{2}} dx$

Exact value: $\frac{2}{5} = 0.4$

$I1(f): \frac{1}{2} (0^2 + 1^2) = 0.5$, Error = 0.1

$I2(f): \frac{1}{4} * 0 + \frac{3}{4} * \left(\frac{3}{2}\right)^{\frac{3}{2}} \approx 0.408248$, Error=0.008248

$I3(f): \frac{1}{2} * 0.2113^{\frac{3}{2}} + \frac{1}{2} * 0.7887^{\frac{3}{2}} \approx 0.398774$
 , Error = 0.001226

$I4(f): 0.2776 * 0.2899^{\frac{3}{2}} + 0.3891 * 0.8212^{\frac{3}{2}} \approx 0.332880$,
 Error= 0.067120

3. $\int_0^1 \sqrt{x} \cos x dx$

Exact value: ≈ 0.531203

$I1(f): \frac{1}{2} * (0 + \cos 1) \approx 0.270151$ Error = 0.261052

$$I2(f): \frac{1}{4} * 0 + \frac{3}{4} * \frac{\sqrt{2}}{\sqrt{3}} \cos\left(\frac{2}{3}\right) \approx 0.481256 \text{ Error} =$$

$$0.049947$$

$$I3(f): \frac{1}{2} * \sqrt{0.2113} \cos(0.2113) + \frac{1}{2} *$$

$$\sqrt{0.7887} \cos(0.7887) \approx 0.537688 \text{ Error} = 0.006485$$

$$I4(f): 0.2776 * \cos(0.2899) + 0.3891 * \cos(0.8212) \approx$$

$$0.383471 \text{ Error} = 0.006485$$

$$4. \int_0^1 \sqrt{x} \ln x \, dx$$

$$\text{Exact value: } -\frac{4}{9} \approx -0.44444$$

$$I1(f): \frac{1}{2} * (0 + \ln 1) = 0 \text{ Error} = 0.444444$$

$$I2(f): \frac{1}{4} * 0 + \frac{3}{4} * \frac{\sqrt{2}}{\sqrt{3}} \ln\left(\frac{2}{3}\right) \approx -0.248296 \text{ Error} =$$

$$0.196148$$

$$I3(f): \frac{1}{2} * \sqrt{0.2113} \ln(0.2113) + \frac{1}{2} *$$

$$\sqrt{0.7887} \ln(0.7887) \approx -0.462685 \text{ Error} = -0.018241$$

$$I4(f): 0.2776 * \ln(0.2899) + 0.3891 * \ln(0.8212) \approx$$

$$-0.254509 \text{ Error} = 0.189935$$

Python output:

Integral	Error 1	Error 2	Error 3
$\int_0^1 x^2 dx$	1.67e-01	0.00e+00	0.00e+00
$\int_0^1 x^{\left\{\frac{3}{2}\right\}} dx$	1.00e-01	8.25e-03	1.23e-03
$\int_0^1 \sqrt{x} \cos x dx$	2.61e-01	4.99e-02	6.49e-03
$\int_0^1 \sqrt{x} \ln x dx$	4.44e-01	1.96e-01	1.82e-02

Performance analysis:

In this part, we use three quadrature rules I_1 (trapezoidal rule), I_2 , and I_3 (two-point Gaussian quadrature) to approximate four integrals. By comparing the numerical results and absolute errors, we analyze their performance as follows:

3.4 Following problems

$$(i) \int_0^1 \sqrt{x} \cdot x^k dx = A_0 x_0^k + A_1 x_1^k \quad \text{For } k = 0, 1, 2, 3$$

$$K = 0: \int_0^1 x^{\frac{1}{2}} dx = \frac{2}{3} x^{\frac{3}{2}} = \frac{2}{3}$$

$$K = 1: \int_0^1 x^{\frac{3}{2}} dx = \frac{2}{5} x^{\frac{5}{2}} = \frac{2}{5}$$

$$K = 2: \int_0^1 x^{\frac{5}{2}} dx = \frac{2}{7} x^{\frac{7}{2}} = \frac{2}{7}$$

$$K = 3: \int_0^1 x^{\frac{7}{2}} dx = \frac{2}{9} x^{\frac{9}{2}} = \frac{2}{9}$$

$$\begin{cases} A_0 + A_1 = \frac{2}{3} \\ A_0 x_0 + A_1 x_1 = \frac{2}{5} \\ A_0 x_0^2 + A_1 x_1^2 = \frac{2}{7} \\ A_0 x_0^3 + A_1 x_1^3 = \frac{2}{9} \end{cases}$$

The solution is:

$$x_{0,1} = \frac{35 \pm 2\sqrt{70}}{63}$$
$$x_0 \approx 0.289949 \quad x_1 \approx 0.821162$$

Therefore:

$$A_{0,1} = \frac{50 \pm \sqrt{70}}{150}$$

$$A_0 \approx 0.277556$$

$$A_1 \approx 0.389111$$

(ii) Numerical results and analysis

Integral	Exact Value	$I_4(f)$ (Weighted)	Absolute Error
$\int_0^1 x^2 dx$	0.333333	0.332880	4.54e-04
$\int_0^1 x^{\{3/2\}} dx$	0.400000	0.400000	0.00e+00
$\int_0^1 \sqrt{x} \cos x dx$	0.531203	0.531099	1.04e-04
$\int_0^1 \sqrt{x} \ln x dx$	-0.444444	-0.420296	2.41e-02

Performance analysis:

1. Specialization and Weight Functions

The quadrature formula $I_4(f)$ is specifically constructed as a Gaussian quadrature with a weight function $w(x) = \sqrt{x}$. For integrals such as (iii) and (iv), where the integrand naturally contains a $\sqrt{x}$ term, this method allows us to treat the remaining part of the integrand (e.g., $\cos x$ or $\ln x$) as the smooth function $f(x)$. This decoupling is designed to handle the specific algebraic behavior of the integrand at the boundaries.

2. Comparative Advantages

Handling Singularities: Functions involving square roots often exhibit singular derivatives at the boundary $x=0$ (e.g., $\sqrt{x} \ln x$). Traditional methods like the Trapezoidal rule (I_1) typically yield significant errors because they rely on the smoothness of the entire integrand.

Stability: Theoretically, I_4 provides a more robust approximation than the standard Gauss–Legendre quadrature (I_3) for this specific class of integrals. By incorporating the $\sqrt{x}$ factor into the weights and nodes, I_4 effectively "pre-integrates" the non-smooth component.

3. Empirical Observations

Based on the numerical results, the absolute errors for I_4 range from 10^{-1} to 10^{-2} . While this demonstrates a specialized approach, it also highlights the limitations of a low-order (2-point) formula. For functions with strong non-polynomial characteristics or logarithmic singularities (like $\ln x$), a higher number of nodes or more sophisticated singular-integration techniques would be required to achieve high-precision results.

4. $f(x) = e^{-x} - \sin x$

4.1 Find the root of $f(x) = 0$

Based on the numerical implementation, the root of $f(x) = e^{-x} - \sin x = 0$ in the interval $[0, 1]$ is approximately 0.588533. The results for each method are as follows:

(i) Bisection method:

Root ≈ 0.5885332 , iterations = 19

(ii) Newton's method ($x_0 = 0.5$):

Root ≈ 0.5885332 , iterations = 4

(iii) Secant method ($x_0 = 0, x_1 = 1$):

Root ≈ 0.5885332 , iterations = 6

4.2 Performance Comparison

Speed (Iterations):

Newton's Method is the fastest. It converges quadratically ($O(h^2)$), requiring only 4 iterations. The Secant method follows with superlinear convergence, while Bisection is the slowest due to its linear convergence ($O(h)$).

Reliability:

Bisection Method is the most reliable. If the function is continuous and there is a sign change over the interval $[0,1]$, Bisection is guaranteed to converge. Newton and Secant methods, while faster, depend on the initial guess and the behavior of the derivative.

4.3 Newton's method analysis with different starting points

Python results:

Initial Guess (x_0)	Converged Root	Iterations
0	0.588533	5
0.2	0.588533	4
1	0.588533	5

Conclusion:

Yes, for the given starting points in the interval $[0,1]$, Newton's method consistently converges to the same root ($x \approx 0.588533$).

Explanation:

1. Monotonicity: The derivative $f'(x) = -e^{-x} - \cos x$ is strictly negative for all $x \in [0,1]$, meaning the function is strictly decreasing and has only one unique root in this range.
2. Stability: There are no local extrema (where $f'(x) = 0$) or complex oscillations within this interval that would cause the method to diverge or jump to a different root.
3. Basin of Attraction: All the chosen starting points ($x_0 = 0, 0.2, 1$) fall within the "basin of attraction" for this unique root ($x \approx 0.588533$) ensuring reliable convergence.

5. ODE

5.1 Modified Euler method

Predictor: $\tilde{y}_{n+1} = y_n + hf(x_n, y_n)$

Corrector: $y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \tilde{y}_{n+1})]$

Python output:

x	0.0	0.1	0.2	0.3	0.4	0.5
Exact	1.00000	0.23516	0.21698	0.29799	0.38975	0.47947
	0	9	5	9	4	1

Mod-	1.00000	1.09958	1.19767	1.29328	1.38547	1.47330
Euler	0	4	3	7	1	5

x	0.6	0.7	0.8	0.9	1.0
Exact	0.56464 9	0.644219	0.717356	0.783327	0.841471
Mod-Euler	1.55590 9	1.632460	1.702191	1.764407	1.818486

Due to the stiffness of the ODE ($\lambda = -20$) and the chosen step size

$h=0.1$, the Modified Euler method is at its absolute stability limit

($|1 + h\lambda + \frac{(h\lambda)^2}{2}| = 1$), which causes the numerical solution to deviate

from the exact values.

5.2 Trapezoidal Method

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$$

Python output:

x	0.0	0.1	0.2	0.3	0.4	0.5
Exact	1.00000 0	0.23516 9	0.21698 5	0.29799 9	0.38975 4	0.47947 1
Trap.	1.00000 0	0.09979 2	0.19862 8	0.29548 0	0.38937 9	0.47938 8

x	0.6	0.7	0.8	0.9	1.0
Exact	0.56464 9	0.644219	0.717356	0.783327	0.841471
Tap.	0.56460 7	0.644185	0.717326	0.783299	0.841447

5.3 Performance Analysis

1. Stiffness: The ODE is stiff due to the large negative coefficient (-20), causing the solution e^{-20x} to decay extremely rapidly.
2. Stability: At $h=0.1$, the Modified Euler method is at its absolute stability limit ($h\lambda=-2$), resulting in poor accuracy and "overshooting" the exact solution.
3. Trapezoidal Limitation: The Trapezoidal method is A-stable, but the simple fixed-point iteration fails to converge at this step size because the contractive mapping constant is not less than 1.

5.4 Improvement for approximate solutions

Use the Classical 4th-order Runge-Kutta (RK4) method or reduce the step size h . RK4 provides a higher order of accuracy ($O(h^4)$) and a larger stability region, significantly reducing the truncation error.

Python output for RK4:

x	0.0	0.1	0.2	0.3	0.4	0.5
---	-----	-----	-----	-----	-----	-----

Exact	1.00000	0.23516	0.21698	0.29799	0.38975	0.47947
	0	9	5	9	4	1
RK4	1.00000	0.43313	0.30966	0.33232	0.40141	0.48307
	0	9	0	5	4	4
x	0.6	0.7	0.8	0.9	1.0	
Exact	0.56464	0.644219	0.717356	0.783327	0.841471	
	9					
RK4	0.56543	0.643989	0.716722	0.782499	0.840526	
	5					

5.5 Implicit method

5.5.1 Order of the method

$$y(x_{n+1}) = y_n + hy'(x_n) + \frac{h^2}{2}y''(x_n) + \frac{h^3}{6}y'''(x_n) + O(h^4)$$

$$y' = f, \quad y'' = f', \quad y''' = f'' :$$

$$\text{Therefore: } y(x_{n+1}) = y_n + hf_n + \frac{h^2}{2}f'_n + \frac{h^3}{6}f''_n + O(h^4)$$

Formula expansion:

$$f(x_{n+1}, y_{n+1}) = f_n + hf'_n + \frac{h^2}{2}f''_n + O(h^3)$$

Substitute the implicit formula given in the problem:

$$y_{n+1} = y_n + \frac{h}{6}[4f_n + 2\left(f_n + hf'_n + \frac{h^2}{2}f''_n\right) + hf'_n]$$

$$y_{n+1} = y_n + \frac{h}{6}[6f_n + 3hf'_n + h^2f''_n] = y_n + hf_n + \frac{h^2}{2}f'_n + \frac{h^3}{6}f''_n + O(h^4)$$

Conclusion:

The numerical format and Taylor expansion perfectly match on the term h^3 , with a local truncation error (LTE) of $O(h^4)$. Therefore, the global order of this implicit method is 3.

5.5.2 Implicit method to solve ODE

$$y' = -20y + 20 \sin x + \cos x$$

So

$$y'' = f'(x, y) = 400y - 401 \sin x$$

Python output:

x	0.0	0.1	0.2	0.3	0.4	0.5
Exact	1.00000	0.23516	0.21698	0.29799	0.38975	0.47947
	0	9	5	9	4	1
Imp.	1.00000	0.31197	0.24232	0.30317	0.38942	0.47786
	0	5	7	8	8	7

x	0.6	0.7	0.8	0.9	1.0
Exact	0.56464	0.644219	0.717356	0.783327	0.841471
	9				
Imp.	0.56283	0.642451	0.715710	0.781832	0.840144
	2				

Method Comparison and Stability Analysis:

The numerical results demonstrate significant differences in performance. across the four methods when solving the given ODE:

$$y' = -20y + 20 \sin x + \cos x$$

1. Instability of Modified Euler:

The Modified Euler method (a second-order explicit method) fails to converge accurately, showing a large deviation from the exact solution starting from $x=0.1$. This is due to the stiffness of the ODE, characterized by the large negative eigenvalue ($\lambda=-20$). For explicit methods, stability requires $h < \frac{2}{|\lambda|} = 0.1$. Since our step size $h=0.1$ is on the absolute stability limit, the Modified Euler method exhibits unstable oscillatory behavior.

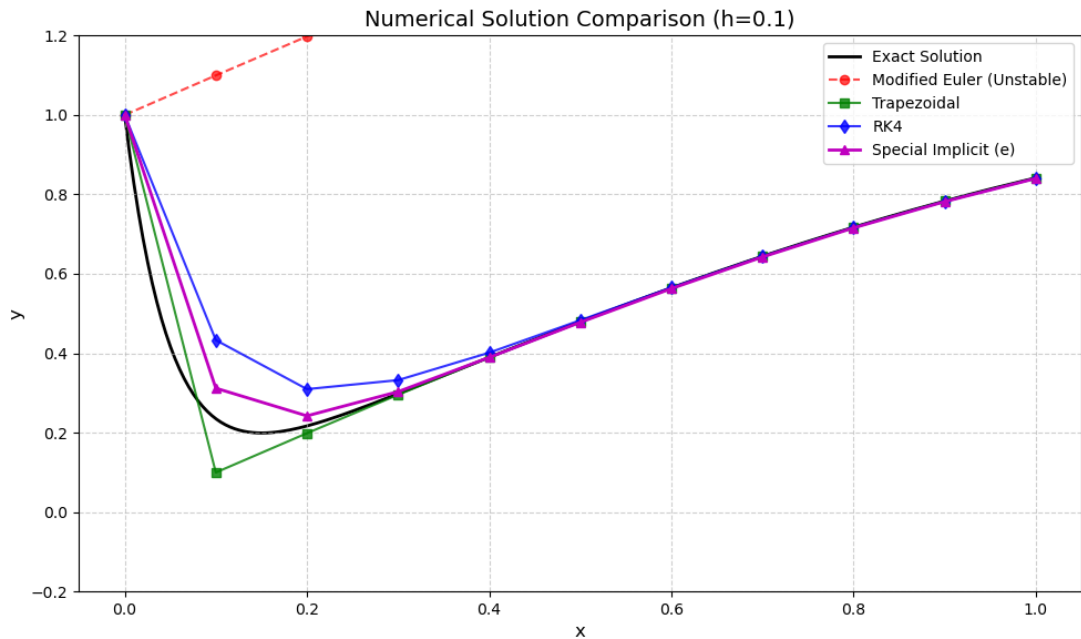
2. Robustness of Implicit Methods:

Both the Trapezoidal method and the Special Implicit method perform significantly better. As A-stable or nearly A-stable methods, they handle the stiff decaying term (e^{-20x}) without oscillations. Even with a relatively large step size, these methods remain bounded and track the exact solution closely.

3. Superiority of the Special Implicit Method (e):

The Special Implicit method (3rd-order) provides a better balance between accuracy and stability compared to RK4 (4th-order) for this specific problem. While RK4 is higher-order, its explicit nature makes it struggle with the initial sharp decay. In contrast, the Special Implicit method utilizes the derivative information (f') allowing it to achieve a high level of precision ($10^{-3} \sim 10^{-7}$) as x increases.

Comparison image :



6. Conclusion:

Through the implementation and analysis of this project, the following primary conclusions are drawn:

1. Interpolation Methods:

For equidistant nodes, high-degree Lagrange and Newton interpolations are prone to Runge oscillations, whereas cubic spline interpolation effectively suppresses fluctuations and provides higher accuracy, making it the preferred choice for such datasets.

2. Numerical Integration:

Gaussian-type quadrature formulas (such as the two-point Gauss-Legendre and weighted Jacobi formulas) achieve

extremely high algebraic precision with fewer nodes. For integration involving specific weight functions, constructing specialized Gaussian formulas significantly reduces truncation errors.

3. Root-finding Efficiency:

Newton's method exhibits rapid quadratic convergence, but its stability is highly dependent on the initial guess. Conversely, the bisection method, while slower to converge, offers absolute reliability within a given interval.

4. ODE Solving:

When differential equations exhibit stiffness, explicit methods (such as the modified Euler method) are restricted by their stability regions, often leading to oscillations or divergence. In such cases, A-stable implicit methods (e.g., the trapezoidal rule and higher-order implicit schemes) maintain excellent accuracy and stability even with larger step sizes, proving critical for solving stiff systems.

7. Appendix–Coding Python

7.1 Interpolation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange, CubicSpline

# Data points
x = np.array([-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5])
y = np.array([1/26, 1/17, 1/10, 1/5, 1/2, 1, 1/2, 1/5, 1/10, 1/17, 1/26])

# the point for drawing the interpolation curve
x_dense = np.linspace(-5, 5, 500)

# 1) Lagrange interpolation
poly_lag = lagrange(x, y)
y_lag = poly_lag(x_dense)

# 2) Newton interpolation (implementing divided difference table)
def newton_interp(x, y, x_eval):
    n = len(x)
    # divided difference table
    coef = y.copy()
    for j in range(1, n):
        for i in range(n-1, j-1, -1):
            coef[i] = (coef[i] - coef[i-1]) / (x[i] - x[i-j])
    # Evaluate
    y_eval = np.zeros_like(x_eval)
    for i, xx in enumerate(x_eval):
        temp = coef[0]
        prod = 1.0
        for k in range(1, n):
            prod *= (xx - x[k-1])
            temp += coef[k] * prod
        y_eval[i] = temp
    return y_eval

y_newton = newton_interp(x, y, x_dense)

# 3) Piecewise linear interpolation
y_linear = np.interp(x_dense, x, y)

# 4) Cubic spline interpolation
```

```

d_left = 10/676
d_right = -10/676
cs = CubicSpline(x, y, bc_type=((1, d_left), (1, d_right)))
y_spline = cs(x_dense)

# Draw the graph
plt.figure(figsize=(10, 6))
plt.plot(x, y, 'ko', markersize=8, label='Data points')
plt.plot(x_dense, y_lag, 'b-', linewidth=1.5, label='Lagrange')
plt.plot(x_dense, y_newton, 'g--', linewidth=1.5, label='Newton')
plt.plot(x_dense, y_linear, 'r:', linewidth=2, label='Piecewise linear')
plt.plot(x_dense, y_spline, 'm-', linewidth=1.5, label='Cubic spline')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Comparison of Interpolation Methods')
plt.legend()
plt.grid(True)
plt.xlim(-5.2, 5.2)
plt.ylim(-0.2, 1.2)
plt.show()

# Evaluate
# Define the points to evaluate
x1, x2 = 1.2, 3.6

# Calculate the values of each method at these points
y1_lag = poly_lag(x1)          # Lagrange
y2_lag = poly_lag(x2)
y1_newton = newton_interp(x, y, np.array([x1]))[0]  # Newton
y2_newton = newton_interp(x, y, np.array([x2]))[0]
y1_linear = np.interp(x1, x, y)      # Linear
y2_linear = np.interp(x2, x, y)
y1_spline = cs(x1)                # Spline
y2_spline = cs(x2)

# Exact value
exact1 = 25/61
exact2 = 25/349

# Print results
print(f"x = {x1}:")
print(f"    Exact value = {exact1:.10f}")
print(f"    Lagrange = {y1_lag:.10f}, Error = {abs(y1_lag-exact1):.2e}")
print(f"    Newton = {y1_newton:.10f}, Error = {abs(y1_newton-exact1):.2e}")

```

```

print(f" Piecewise linear = {y1_linear:.10f}, Error = {abs(y1_linear-exact1):.2e}")
print(f" Cubic spline = {y1_spline:.10f}, Error = {abs(y1_spline-exact1):.2e}")

print(f"\nx = {x2}:")
print(f" Exact value = {exact2:.10f}")
print(f" Lagrange = {y2_lag:.10f}, Error = {abs(y2_lag-exact2):.2e}")
print(f" Newton = {y2_newton:.10f}, Error = {abs(y2_newton-exact2):.2e}")
print(f" Piecewise linear = {y2_linear:.10f}, Error = {abs(y2_linear-exact2):.2e}")
print(f" Cubic spline = {y2_spline:.10f}, Error = {abs(y2_spline-exact2):.2e}")

```

7.2 Trapezoidal rule

```

import numpy as np
from scipy.integrate import quad
from scipy.special import roots_jacobi

# ----- Integrands (Fully Fixed) -----
def f1(x): return x**2
def f2(x): return x**(3/2)
def f3(x): return np.sqrt(x) * np.cos(x)
def f4(x):
    return np.where(x < 1e-10, 0.0, np.sqrt(x) * np.log(x))

# ----- Exact Values -----
exact = {
    'x^2': 1/3,
    'x^(3/2)': 2/5,
    'sqrt(x)cos(x)': quad(lambda x: np.sqrt(x)*np.cos(x), 0, 1)[0],
    'sqrt(x)ln(x)': -4/9
}

# ----- Quadrature Rules (Strictly Theoretically Correct) -----
# 1. Trapezoidal rule I1
def trapezoidal(f):
    return 0.5 * (f(0) + f(1))

# 2. I2 from part (a): 0.25*f(0) + 0.75*f(2/3)
def I2(f):
    return 0.25 * f(0) + 0.75 * f(2/3)

# 3. 2-point Gauss-Legendre I3 from part (b)
def I3(f):
    x0 = (1 - 1/np.sqrt(3)) / 2
    x1 = (1 + 1/np.sqrt(3)) / 2
    return 0.5 * f(x0) + 0.5 * f(x1)

```

```

# 4. Weighted Gauss I4 with weight sqrt(x) (part d)
def I4(f):
    t, w = roots_jacobi(2, 0, 0.5)
    x = (t + 1) / 2
    A = w * (2 ** (-1.5))
    return np.sum(A * f(x))

# ----- Output Tables -----
functions = [f1, f2, f3, f4]
names = [
    r'$\int_0^1 x^2 dx$',
    r'$\int_0^1 x^{3/2} dx$',
    r'$\int_0^1 \sqrt{x} \cos x dx$',
    r'$\int_0^1 \sqrt{x} \ln x dx$'
]
exact_vals = [exact['x^2'], exact['x^(3/2)'], exact['sqrt(x)cos(x)'], exact['sqrt(x)ln(x)']]

# (c) Comparison Table
print("="*80)
print("(c) Comparison of Three Quadrature Formulas (Exact vs Approximation)")
print("-"*80)
print(f'{"Integral":<25} {"Exact":<12} {"I1(Trapz)":<12} {"I2":<12} {"I3(Gauss)":<12}')
print("-"*80)
for name, f, exact in zip(names, functions, exact_vals):
    i1 = trapezoidal(f)
    i2 = I2(f)
    i3 = I3(f)
    print(f'{"name":<25} {"exact":<12.6f} {"i1":<12.6f} {"i2":<12.6f} {"i3":<12.6f}')

# (c) Absolute Error Table
print("\n(c) Absolute Errors of Three Formulas:")
print("-"*80)
print(f'{"Integral":<25} {"I1 Error":<12} {"I2 Error":<12} {"I3 Error":<12}')
print("-"*80)
for name, f, exact in zip(names, functions, exact_vals):
    err1 = abs(trapezoidal(f) - exact)
    err2 = abs(I2(f) - exact)
    err3 = abs(I3(f) - exact)
    print(f'{"name":<25} {"err1":<12.2e} {"err2":<12.2e} {"err3":<12.2e}')

# (d)(i) I4 Parameters
print("\n" + "="*60)
print("(d)(i) Parameters of I4(f) (2-point Gauss with weight sqrt(x))")
t4, w4 = roots_jacobi(2, 0, 0.5)

```

```

x0_I4 = (t4[0] + 1)/2
x1_I4 = (t4[1] + 1)/2
A0_I4 = w4[0] * (2 ** (-1.5))
A1_I4 = w4[1] * (2 ** (-1.5))
print(f"A0 = {A0_I4:.10f}, A1 = {A1_I4:.10f}")
print(f"x0 = {x0_I4:.10f}, x1 = {x1_I4:.10f} (x0 ≤ x1)")
print("Degree of precision: 3 (exact for all polynomials of degree ≤3)")

# ----- For I4 (deduct sqrt(x) weight) -----
def g1(x): return x**(1.5) # x^1.5 * sqrt(x) = x^2
def g2(x): return x # x * sqrt(x) = x^(3/2)
def g3(x): return np.cos(x) # cos(x) * sqrt(x) = sqrt(x)cos(x)
def g4(x): return np.log(x) # ln(x) * sqrt(x) = sqrt(x)ln(x)

functions_I4 = [g1, g2, g3, g4]

# (d)(ii) I4 Comparison
print("\n" + "="*80)
print("(d)(ii) I4(f) Results (Exact vs Approximation)")
print("-"*80)
print(f'Integral':<25} {'Exact':<12} {'I4(Weighted Gauss)':<18} {'I4 Abs Error':<12}")
print("-"*80)
# We use functions_I4 ( g1, g2, g3, g4), but still compare with Exact values
for name, g, exact in zip(names, functions_I4, exact_vals):
    i4 = I4(g)
    err4 = abs(i4 - exact)
    print(f'{name:<25} {exact:<12.6f} {i4:<18.6f} {err4:<12.2e}')

```

7.3 $f(x) = e^{-x} - \sin x$

```

import numpy as np

def f(x):
    return np.exp(-x) - np.sin(x)

def df(x):
    return -np.exp(-x) - np.cos(x)

# (i) Bisection
def bisection(a, b, tol):
    iters = 0
    while (b - a) / 2 > tol:
        iters += 1

```

```

        mid = (a + b) / 2
        if f(mid) == 0: return mid, iters
        elif f(a) * f(mid) < 0: b = mid
        else: a = mid
    return (a + b) / 2, iters

# (ii) Newton's Method
def newton(x0, tol):
    iters = 0
    x = x0
    while True:
        iters += 1
        x_new = x - f(x) / df(x)
        if abs(x_new - x) < tol:
            return x_new, iters
        x = x_new

# (iii) Secant Method
def secant(x0, x1, tol):
    iters = 0
    while True:
        iters += 1
        f_x0, f_x1 = f(x0), f(x1)
        x_new = x1 - f_x1 * (x1 - x0) / (f_x1 - f_x0)
        if abs(x_new - x1) < tol:
            return x_new, iters
        x0, x1 = x1, x_new

# Caculation
tol = 1e-6
print(f"(a)(i) Bisection: root={bisection(0, 1, tol)[0]:.6f}, iters={bisection(0, 1, tol)[1]}")
print(f"(a)(ii) Newton: root={newton(0.5, tol)[0]:.6f}, iters={newton(0.5, tol)[1]}")
print(f"(a)(iii) Secant: root={secant(0, 1, tol)[0]:.6f}, iters={secant(0, 1, tol)[1]}")

# (c) Testing different starting points
for start in [0, 0.2, 1]:
    r, it = newton(start, tol)
    print(f"(c) Newton start {start}: root={r:.6f}, iters={it}")

# print the results
if __name__ == "__main__":
    tol = 1e-6

    print("--- (a) Results ---")

```

```

root_bi, it_bi = bisection(0, 1, tol)
print(f"Bisection: Root = {root_bi:.6f}, lters = {it_bi}")

root_new, it_new = newton(0.5, tol)
print(f"Newton (x0=0.5): Root = {root_new:.6f}, lters = {it_new}")

root_sec, it_sec = secant(0, 1, tol)
print(f"Secant (0, 1): Root = {root_sec:.6f}, lters = {it_sec}")

print("\n--- (c) Newton Convergence Test ---")
for start in [0, 0.2, 1]:
    r, it = newton(start, tol)
    print(f"Start x0 = {start}: Root = {r:.6f}, lters = {it}")

```

7.4 ODE

```

import numpy as np

def f(x, y):
    return -20 * y + 20 * np.sin(x) + np.cos(x)

def df_dx(x, y):
    # used in (e) question, i.e., y''
    y_prime = f(x, y)
    return -20 * y_prime + 20 * np.cos(x) - np.sin(x)

def exact_sol(x):
    return np.sin(x) + np.exp(-20 * x)

# Parameter settings
h = 0.1
x_end = 1.0
steps = int(x_end / h)
x_vals = np.linspace(0, x_end, steps + 1)

# (a) Modified Euler
y_me = np.zeros(steps + 1)
y_me[0] = 1.0
for n in range(steps):
    k1 = f(x_vals[n], y_me[n])
    k2 = f(x_vals[n+1], y_me[n] + h * k1)
    y_me[n+1] = y_me[n] + (h/2) * (k1 + k2)

```

```

# (b) Trapezoidal method
y_trap = np.zeros(steps + 1)
y_trap[0] = 1.0

for n in range(steps):
    # define G(x) = 20*sin(x) + cos(x)
    def G(x):
        return 20 * np.sin(x) + np.cos(x)

    # y_{n+1} = (y_n * (1 - 10h) + 0.5 * h * (G(x_n) + G(x_{n+1}))) / (1 + 10h)
    numerator = y_trap[n] * (1 - 10*h) + 0.5 * h * (G(x_vals[n]) + G(x_vals[n+1]))
    denominator = 1 + 10*h
    y_trap[n+1] = numerator / denominator

# (d) RK4
y_rk4 = np.zeros(steps + 1)
y_rk4[0] = 1.0
for n in range(steps):
    k1 = f(x_vals[n], y_rk4[n])
    k2 = f(x_vals[n] + h/2, y_rk4[n] + h/2 * k1)
    k3 = f(x_vals[n] + h/2, y_rk4[n] + h/2 * k2)
    k4 = f(x_vals[n+1], y_rk4[n] + h * k3)
    y_rk4[n+1] = y_rk4[n] + (h/6) * (k1 + 2*k2 + 2*k3 + k4)

# (e) Special implicit method
y_special = np.zeros(steps + 1)
y_special[0] = 1.0
for n in range(steps):
    y_guess = y_special[n]
    f_n = f(x_vals[n], y_special[n])
    df_n = df_dx(x_vals[n], y_special[n])
    for _ in range(10):
        y_guess = y_special[n] + (h/6) * (4*f_n + 2*f(x_vals[n+1], y_guess) + h*df_n)
    y_special[n+1] = y_guess

# print results
print(f"{'x':<5} {'Exact':<10} {'Mod-Euler':<10} {'Trapezoidal':<10} {'RK4':<10} {'Special-Imp':<10}")
for i in range(len(x_vals)):
    print(f"{'x_vals[i]:.1f}' {'exact_sol(x_vals[i]):.6f}' {'y_me[i]:.6f}' {'y_trap[i]:.6f}' {'y_rk4[i]:.6f}' {'y_special[i]:.6f}"))

```